School Management System

BATCH NO : JFS-ONR -005

NAME : THARUN GUNDLAPALLY

School Management System

SQL Database Project

Prepared by Tharun

1. Executive Summary

The School Management System is a comprehensive, normalized relational database designed to track and manage student academic performance. By establishing a scalable architecture, this system streamlines the logging of student personal details, curriculum subjects, specific evaluation cycles (exams), and individual grades.

The primary objective of this project is to create a centralized data framework that eliminates redundancy, ensures data integrity across related tables, and allows educators or administrators to execute complex analytical queries. Through advanced SQL operations—such as multi-table joins, aggregations, and conditional logic—the system provides instant insights into class averages, top performers, and overall pass/fail metrics.

2. Core Modules & Architecture

The database is organized into four interconnected modules to maintain a clean Entity-Relationship (ER) structure:

- • Students Table: Acts as the primary dimension table for student demographics, storing unique identifiers, first names, and last names.
- • Subjects Table: A dimension table cataloging the standardized curriculum (e.g., Mathematics, Science, English).
- • Exams Table: Records specific testing events and their corresponding dates.
- • Marks Table (Transaction Hub): The central fact table that links Students, Subjects, and Exams together using Foreign Keys. It stores the numerical scores achieved in each specific instance.

Database Schema Overview

Students: student_id (PK), first_name, last_name
Subjects: subject_id (PK), subject_name
Exams: exam_id (PK), exam_name, exam_date
Marks: mark_id (PK), student_id (FK), subject_id (FK), exam_id (FK), score

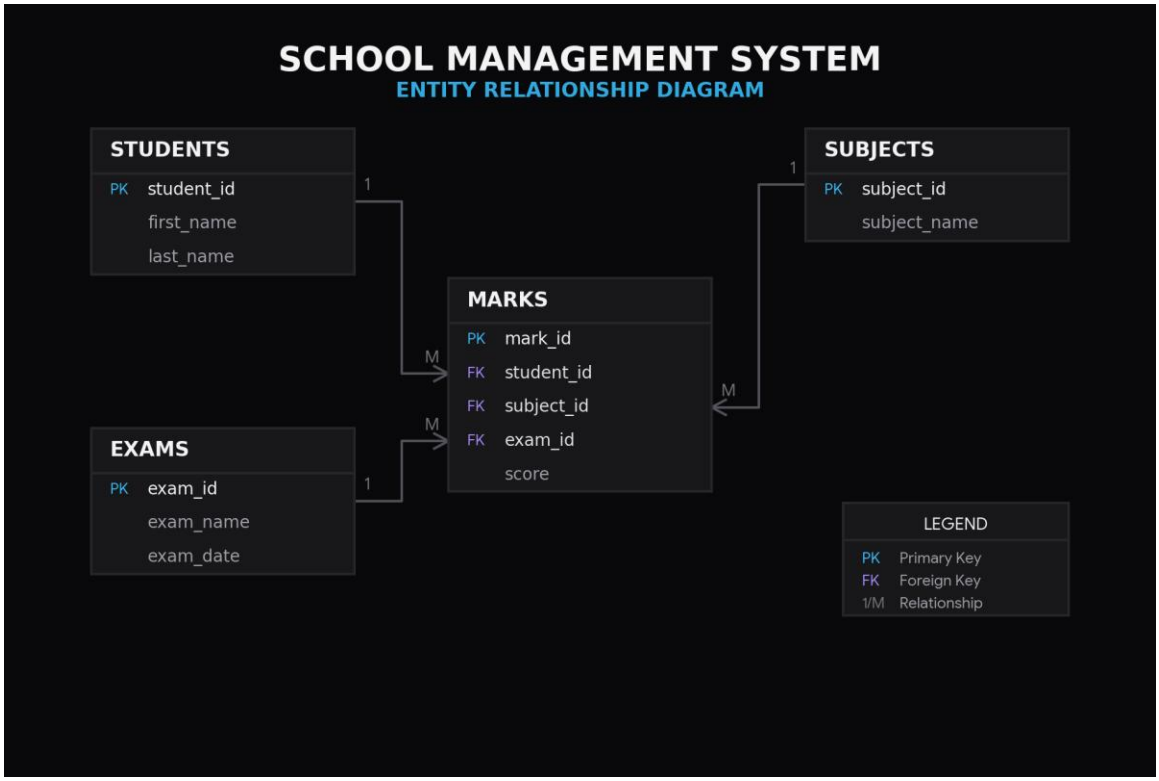


Figure 1: Entity Relationship Diagram (System Architecture)

3. Database Initialization (DDL)

The following Data Definition Language (DDL) scripts were used to structure the relational database:

```
CREATE DATABASE school_management_system;
USE school_management_system;

CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    first_name VARCHAR(50),
    last_name VARCHAR(50)
);

CREATE TABLE Subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(100)
);

CREATE TABLE Exams (
    exam_id INT PRIMARY KEY,
    exam_name VARCHAR(100),
```

```
    exam_date DATE
);

CREATE TABLE Marks (
    mark_id INT PRIMARY KEY,
    student_id INT,
    subject_id INT,
    exam_id INT,
    score INT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id),
    FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);
```

4. Sample Data Integration

The tables were populated with a verified dataset to simulate a live academic environment. Below are the records successfully imported into the system.

STUDENTS TABLE

student_id	first_name	last_name
1	Yadagiri	Goud
2	Narsimha	Roa
3	Mallaiah	Yadav
4	Padma	Patil
5	Laxman	Konda
6	Vikram	Singh
7	Ramesh	Kumar
8	Sunitha	Reddy
9	Kiran	Chary
10	Chandraiah	Varma

SUBJECTS TABLE

subject_id	subject_name
1	Mathematics
2	Science
3	English
4	Hindi
5	Social Studies

EXAMS TABLE

exam_id	exam_name	exam_date
1	Half-Yearly	2024-09-15
2	Annual	2024-03-20

MARKS TABLE (Transaction Records)

mark_id	student_id	subject_id	exam_id	score
1	1	1	1	95
2	1	2	1	62
3	1	3	1	56
4	1	4	1	72
5	1	5	1	70
6	2	1	1	69
7	2	2	1	63
8	2	3	1	61
9	2	4	1	98
10	2	5	1	89
11	3	1	1	60

12	3	2	1	92
13	3	3	1	82
14	3	4	1	57
15	3	5	1	56
16	4	1	1	60
17	4	2	1	68
18	4	3	1	69
19	4	4	1	87
20	4	5	1	93
21	5	1	1	56
22	5	2	1	90
23	5	3	1	67
24	5	4	1	96
25	5	5	1	89
26	6	1	1	81
27	6	2	1	69
28	6	3	1	83
29	6	4	1	92
30	6	5	1	72
31	7	1	1	55

5. Analytical SQL Queries

To extract actionable insights from the database, the following analytical queries were developed utilizing joins, aggregations, and conditional filtering.

1. Retrieve all students

```
SELECT first_name, last_name FROM Students;
```

2. Filter by Score > 80

```
SELECT * FROM Marks WHERE score > 80;
```

3. Sort Exams by Date

```
SELECT * FROM Exams ORDER BY exam_date DESC;
```

4. Student Marks Details

```
SELECT st.first_name, su.subject_name, m.score FROM Marks m JOIN
Students st ON m.student_id = st.student_id JOIN Subjects su ON
m.subject_id = su.subject_id;
```

5. Subject Averages

```
SELECT su.subject_name, ROUND(AVG(m.score), 2) AS average_score
FROM Marks m JOIN Subjects su ON m.subject_id = su.subject_id
GROUP BY su.subject_name;
```

6. Count Marks for Yadagiri Goud

```
SELECT COUNT(m.mark_id) FROM Marks m JOIN Students st ON
m.student_id = st.student_id WHERE st.first_name = 'Yadagiri' AND
st.last_name = 'Goud';
```

7. Top Performer (Highest Single Score)

```
SELECT st.first_name, st.last_name, m.score FROM Marks m JOIN
Students st ON m.student_id = st.student_id ORDER BY m.score DESC
LIMIT 1;
```

8. Pass/Fail Label

```
SELECT st.first_name, su.subject_name, m.score, CASE WHEN m.score
>= 70 THEN 'Pass' ELSE 'Fail' END AS status FROM Marks m JOIN
Students st ON m.student_id = st.student_id JOIN Subjects su ON
m.subject_id = su.subject_id;
```

9. Total Score per Student

```
SELECT st.first_name, SUM(m.score) AS total_score FROM Marks m
JOIN Students st ON m.student_id=st.student_id GROUP BY
st.first_name;
```

10. Subjects with Average Score < 75

```
SELECT su.subject_name, AVG(m.score) AS avg_score FROM Marks m
JOIN Subjects su ON m.subject_id=su.subject_id GROUP BY
su.subject_name HAVING AVG(m.score) < 75;
```